School of Industrial & Intelligent Systems Engineering
The Iby and Aladar Fleischman Faculty of Engineering
Tel Aviv University

Iby and Aladar Fleischman Faculty of Engineering
Tel Aviv University
הפקולטה להנדסה ע"ש איבי ואלדר פליישמן אוניברסיטת תל-אביב

בית הספר להנדסת תעשייה ומערכות נבונות
הפקולטה להנדסה ע"ש איבי ואלדר פליישמן
אוניברסיטת תל אביב

Shuttle Bus Vehicle & Driver Scheduling

Term Project: Service Systems (Spring 2026)

Issued: 14 May 2026 | Due: 18 June 2026

At a glance. You will model, solve, and submit solutions to a combined *vehicle-and-driver scheduling* problem for a shuttle-bus operator running back-and-forth service between two terminals. Your solver may be exact (e.g. MILP) or heuristic; it will be evaluated on correctness (whether it passes the feasibility checker) and on the total cost achieved on a set of benchmark instances.

1 Operational Setting

An operator runs a shuttle-bus service on a back-and-forth schedule between two terminals, A and B. All vehicles are garaged at a central depot d, where drivers also begin and end their working day.

The service operates from **05:00 to 01:00** the next morning (a 20-hour operating window). For each operating day, you are given:

- a *timetable* of every service departure from A and from B (each entry has an origin terminal and a scheduled departure time),
- a conservative, *time-of-day-dependent* travel-time table between the two terminals and between the depot and each terminal.

You must plan:

1. how many vehicles to deploy and the sequence of trips for each,
2. how many drivers to hire and each driver's shift (start, end, meal break), and
3. which driver–vehicle pair operates each scheduled trip.

1.1 Vehicle Rules

Each vehicle starts its day at the depot, performs a sequence of deadhead and service trips, and returns to the depot by the end of service. **Collectively, the fleet must cover every trip in the published timetable, and each trip must be performed by exactly one vehicle.** Movement rules:

- A vehicle moves only while a driver is assigned to it.
- *Service trips* always go $A \leftrightarrow B$, depart at their scheduled departure time, and take the time-dependent travel time of the departure time-of-day.

- *Deadhead trips* only go $d \leftrightarrow A$ or $d \leftrightarrow B$ (i.e. **no empty inter-terminal moves**); their duration is likewise the tabulated time-dependent travel time at the moment the deadhead begins.
- Dwelling at a terminal is permitted between trips, but at any moment no more than **3 vehicles may dwell at a single terminal**. Depot capacity is unlimited.
- The same physical vehicle may be shared by multiple drivers on different (non-overlapping) shifts. When handed over, the vehicle must return to the depot between shifts.

1.2 Driver Rules

A driver is hired for a single shift per day with the following constraints:

- Shift length $L \in [8, 11]$ hours. Furthermore, a driver's shift must **start at a quarter-hour mark** (e.g., 08:00, 08:15, 08:30) and its **duration must be a multiple of 15 minutes** (hence, it also ends at a quarter-hour mark).
- **A driver operates exactly one vehicle for the entire shift**; switching to a different vehicle mid-shift is not permitted. (Different drivers may still reuse the same physical vehicle on non-overlapping shifts; see the vehicle rules above.)
- The shift starts *and* ends at the depot. A driver may also deadhead back to the depot in the middle of the day (e.g. to hand over the vehicle to the next driver at the end of their own shift).
- One mandatory **meal break of at least 1 hour** whose start is at least 3 hours after the shift start *and* whose end is at least 3 hours before the end of the shift. The break may be taken at terminal A, terminal B, or the depot.
- In addition to the meal break, the driver may accumulate short idle waiting periods between trips at their current location.
- Labour cost is a fixed hourly wage c^{reg} for the first 8 hours and an overtime wage $c^{\text{ot}} > c^{\text{reg}}$ for any time between the 8th and 11th hour. A driver paid for L hours ($8 \leq L \leq 11$) costs $8\,c^{\text{reg}} + (L-8)\,c^{\text{ot}}$.

1.3 Objective

Minimise the total daily cost, which consists of three components:

1. **Fixed vehicle cost** c^{fix} per distinct physical vehicle used.
2. **Variable vehicle cost** c^{var} per minute spent on depot–terminal deadhead trips (service trips between terminals are mandatory and do not enter the cost).
3. **Driver cost**, as defined above.

2 Notation

Sets.	$\mathcal{L} = \{A, B\}$	the two service terminals
	d	the depot
	$\mathcal{T}$	scheduled service trips, indexed by t
	$\mathcal{K}$	duties (driver–vehicle pairings on one shift), indexed by k

Trip data. For each $t \in \mathcal{T}$: origin $o_t \in \mathcal{L}$, destination $\bar{o}_t \in \mathcal{L} \setminus \{o_t\}$, scheduled departure time τ_t (minutes after 00:00).

Travel time. $\theta(i, j, s)$ is the travel time from location i to location j when departing at time s, given as a step function of s.

Parameters.

$L_{\min} = 8,\ L_{\max} = 11$	shift-length bounds (h)
$B = 1$	minimum meal-break length (h)
$\alpha = 3,\ \beta = 3$	minimum spacing of break from shift start / end (h)
$C = 3$	terminal capacity (vehicles dwelling)
$c^{\text{fix}}, c^{\text{var}}$	fixed and variable vehicle costs
$c^{\text{reg}}, c^{\text{ot}}$	regular and overtime hourly wages

All concrete numeric values are provided per-instance in the input file (see Section 3), so your code must read them rather than hard-code them.

3 Input (Instance) Format

Each instance is a single JSON file with three top-level keys: `parameters`, `trips`, and `travel_time`. Times throughout are integers in minutes since 00:00; values above 1440 refer to the following morning (e.g. 1500 is 01:00 the next day). The `break_length_hours` parameter denotes the minimum required meal-break duration.

```
{
  "parameters": {
    "service_start_min": 300,
    "service_end_min":  1500,
    "shift_min_hours":  8,
    "shift_max_hours":  11,
    "break_length_hours": 1,
    "break_min_from_start_hours": 3,
    "break_min_from_end_hours":   3,
    "terminal_capacity": 3,
    "cost_fixed_vehicle":        500,
    "cost_variable_per_min":       1,
    "cost_driver_regular_per_h":  50,
    "cost_driver_overtime_per_h": 75
  },
  "trips": [
    {"trip_id": 1, "origin": "A", "destination": "B", "departure_min": 300},
    {"trip_id": 2, "origin": "B", "destination": "A", "departure_min": 305}
  ],
  "travel_time": {
    "legs": ["A-B","B-A","D-A","A-D","D-B","B-D"],
    "buckets": [
      {"from_min":   0, "to_min":  360, "minutes": [25,25,15,15,20,20]},
      {"from_min": 360, "to_min":  540, "minutes": [40,40,25,25,30,30]}
    ]
  }
}
```

A bucket applies to any *departure* time s with `from_min` $\leq s <$ `to_min`. Buckets must cover the whole service window contiguously. "D" in the leg list denotes the depot.

4 Output (Solution) Format

A solution is a single JSON file describing one `duties` list. Each duty is one driver's shift driving one vehicle. If a vehicle is handed over to a different driver later in the day, two different duties share the same `vehicle_id`.

```
{
  "instance_id": "small_01",
  "duties": [
    {
```

```
      "duty_id":   "k1",
      "driver_id": "d1",
      "vehicle_id":"v1",
      "shift_start_min": 270,
      "shift_end_min":   810,
      "break_start_min": 540,
      "break_end_min":   600,
      "break_location":  "A",
      "activities": [
        {"type":"deadhead","from":"D","to":"A","start_min":270,"end_min":300},
        {"type":"service","trip_id":1,"start_min":300,"end_min":340},
        {"type":"wait","at":"B","start_min":340,"end_min":405},
        {"type":"service","trip_id":8,"start_min":405,"end_min":445},
        ...
        {"type":"break","at":"A","start_min":540,"end_min":600},
        ...
        {"type":"deadhead","from":"A","to":"D","start_min":795,"end_min":810}
      ]
    }
  ]
}
```

Activity types.

- `deadhead`: depot–terminal move, `from` and `to` in $\{ D, A, B \}$ with exactly one side equal to D. Duration must equal the tabulated travel time at `start_min`.
- `service`: covers one scheduled trip. `start_min` must equal τ_t; duration must equal the tabulated $A \leftrightarrow B$ travel time at τ_t.
- `wait`: idle at a single location $\{ D, A, B \}$; duration ≥ 0.
- `break`: a single meal break of at least 60 minutes, listed both in `activities` and mirrored by the duty-level `break_start_min`/`break_end_min` fields.

Activity-sequence rules within a duty.

1. The first activity is a `deadhead` starting at the depot and beginning at `shift_start_min`.
2. The last activity is a `deadhead` ending at the depot and finishing at `shift_end_min`.
3. Consecutive activities are time-contiguous: the end of one equals the start of the next.
4. The end location of an activity equals the start location of the next activity (with `wait`/`break` staying at a single location).
5. Exactly one `break` activity appears, with duration ≥ 60 min.

5 Feasibility & Cost Checker

A reference Python checker, `check_solution.py`, accompanies this document. Usage:

```
python check_solution.py  instance.json  solution.json
```

It prints `OK` and the cost breakdown on success, or a list of violated constraints (with line/duty references) on failure. Your submission must pass this checker on every benchmark instance. You are encouraged to call the checker inside your own pipeline during development.

A `solution_schema.json` is also provided; your solution files must validate against it.

6 Deliverables

Submit a single archive (`.zip`) named `ssp2026_<id1_id2_id3>.zip` containing:

1. **Source code** of your solver (Python or C++; any external libraries must be freely available and documented in a `README.md`).
2. A `README.md` with build/run instructions and the hardware and wall-clock time used to produce each solution.
3. **Solution files** `solution_<instance>.json` for every *released* benchmark instance (`small_02`, `medium_01`, `medium_02`, `large_01`). The grader will additionally run your code on the two hidden evaluation instances.
4. A **written report** (PDF, **at most 6 pages**, in *English or Hebrew*). The report must **not repeat the problem statement**; devote it entirely to **(a)** a clear description of your solution method (modelling choices, algorithm, parameter tuning) and **(b)** a summary of your computational results (per-instance objective value, runtime, and anything notable).

7 Submission Policy

Penalty for infeasible or misreported submissions. For any instance, a submission that includes (i) a solution file that **fails the reference checker** or (ii) a solution accompanied by an **objective value that does not match** the value computed by the checker on the same solution file will incur a grade penalty for that instance that is **strictly larger than simply not submitting a solution for the instance at all**.

This policy is designed to reward honest, well-tested submissions. A small bug that causes the checker to reject your file is far less costly to your grade than pretending the file is feasible when it is not.

Runtime limit. Short running times are part of the design goals of this project. For evaluation purposes, your submitted code must produce its solution for one benchmark instance within **five (5) minutes of wall-clock time** on the teaching-lab machine (one process, no network access). A run that does not terminate within this budget will be killed, and any output written after the five-minute mark is discarded and treated as *no submission* for that instance.

8 Suggested Approaches

You may take *any* approach that produces feasible solutions. Natural starting points include:

- **Sequential VSP → CSP.** First solve a Vehicle Scheduling Problem that packs trips into vehicle blocks honouring the terminal capacity; then cover each vehicle's block with driver duties.
- **Integrated MILP.** A time-space network formulation with indicator variables for each duty's start time, break time and vehicle. Tractable only on small instances.
- **ILP with set-partitioning formulation.** Enumerate all feasible duties, or a promising subset of them, and use a mathematical solver to identify a minimum-cost subset that covers all trips.
- **Metaheuristics.** Large-neighbourhood search, GRASP, tabu search, genetic algorithms, or simulated annealing over solution encodings.
- **Any other method that you devise**, as long as it produces feasible solutions.

You are encouraged (but not required) to compare two methods in your report.

9 Benchmark Instances

Released on 14 May 2026:

- `small_01`: small debugging instance released with a feasible (but suboptimal) reference solution.
- `small_02`: small debugging instance for which you should submit a feasible solution by June 4.
- `medium_01`: realistically sized instance that you may use to validate your pipeline and intuition.

Released on 4 June 2026: Two realistic medium- and large-sized instances: `medium_02` and `large_01`.

Hidden evaluation instances. Two further instances, `medium_03` and `large_02`, are withheld until grading; the grader runs your submitted code on them under the five-minute wall-clock limit. Quality is scored across all benchmark instances (released and hidden) so that *both* families of methods can shine: a well-tuned exact approach (e.g. MILP within the five-minute budget) can dominate on the small instances and perhaps the medium-sized ones, while a thoughtful heuristic is likely required to produce a good solution on the larger ones. If you submit two solvers, the grader will run both on each instance and will evaluate your work based on the better solution obtained.

10 Grading

Criterion	Weight	Notes
Feasibility (`check_solution.py` passes)	35%	scored per instance; an infeasible or misreported submission is penalised below a missing submission (see §7).
Solution quality (gap to best-known value)	30%	scaled by the best feasible value submitted across all groups.
Algorithmic design & correctness	15%	as presented in the report and code.
Written report	15%	clarity and depth of the method description and results; 6 pages max.
Code quality & reproducibility	5%	clean structure, a working `README`.
Bonus	+5%	For submitting a feasible solution for `medium_02` by June 11; scaled by the best feasible solution submitted across all groups.

11 Rules & Academic Integrity

- **Work in teams of up to three students.** All team members must understand every part of the submission and are graded jointly.
- You may use off-the-shelf LP/MILP/CP solvers (e.g. Gurobi, CPLEX, HiGHS, OR-Tools).
- Sharing code between teams is prohibited. Discussing ideas at a high level is permitted.
- AI assistants (including Claude, ChatGPT, Gemini, and Copilot) may be used for coding, debugging, and writing help, but the mathematical formulation and core ideas of the algorithm must be yours. Disclose any AI use in a short statement at the end of the report.

Milestones

14 May 2026	Official project release date; teams of up to three students are formed; `small_01`, `small_02`, and `medium_01` distributed.
4 June 2026	Midpoint checkpoint: your solver produces a feasible, checker-approved solution on both small instances. Submission guidelines for these solutions will be published. On the same day, `medium_02` and `large_01` are released.
11 June 2026	Optional deadline to submit your solution for `medium_02` to earn a bonus.
18 June 2026	Final submission deadline (code, solution files, `results.csv`, and 6-page report).

Good luck!

Acknowledgement: Thanks to Dr. Michal Masin of Optibus (https://optibus.com/) for providing the idea for this assignment.